CS453 Assignment 3 write-up

sandbox1

We discover that the syscalls `write` and `ioctl` are blocked by seccomp sandbox.
Hence we must use an alternative write syscall not blocked by seccomp , i.e`writev()` syscall.
This allows us to write a program that simply invokes:

1. Open the flag file using `open()`
2. Read the contents of the flag file using `read()`
3. Write the contents of the flag file to stdout.

Compile Command:

```
gcc -o sploit1 sploit1.c
```

sandbox2

We discover that only the syscalls listed below are allowed by seccomp:

1. `read()`
2. `write()`
3. `open()`
4. `close()`
5. `fstat()`
6. `mmap()`
7. `mprotect()`
8. `munmap()`
9. `brk()`
10. `pread64()`
11. `access()`
12. `execve()`
13. `uname()`
14. `readlink()`
15. `arch_parctl()`
16. `set_tid_address()`
17. `exit_group()`
18. `newfstatat()`
19. `set_robust_list()`
20. `prlimit64()`
21. `getrandom()`
22. `0x14e`, which in x86 architecture is `pwritev()`

In order to ensure our sploit2 program does not invoke additional syscalls other than those allowed, we have to must avoid any C library startup routines, which might invoke restricted syscalls like `futex()`. To do this

we need to create a custom _start and use a custom syscall wrapper.
To ensure that the dynamic loader is not invoked, we must compile `sploit2.c` with `-static` and `-nostdlib` flags.
Compile command:

```
gcc -static -nostdlib -o sploit2 sploit2.c
```

sandbox3

We discovered that the seccomp filter in sandbox3 blocks nearly every standard output syscall, such as `write()`, `writev()`, `pwrite64()`, `sendfile()`, etc. The syscalls for reading files (e.g. `openat()`, `read()`, and `close()`) are allowed. To bypass the restrictions on output, our approach is to leverage the kernel's IA-32 (32-bit) compatibility:

1. **Memory Allocation in the Low 32-bit Region:**
   We use the 64-bit `mmap()` syscall with the `MAP_32BIT` flag. This forces the allocation of a buffer in the lower 2GB of address space, ensuring that its pointer fits within 32 bits. This is required for passing it to a 32-bit syscall.

2. **Allowed File I/O via 64-bit Syscalls:**
   We use allowed 64-bit syscalls (`openat()`, `read()`, and `close()`) to open the flag file and load its contents into the buffer.

3. **Bypassing Output Restrictions with a 32-bit Write:**
   Since standard 64-bit output syscalls are blocked, we invoke the 32-bit `write` syscall using the legacy `int 0x80` instruction.

   - We implement this in inline assembly by casting our file descriptor and buffer pointer to 32-bit values and then moving them into the correct 32-bit registers before issuing `int 0x80`.
   - This method uses the 32-bit syscall number for `write` (which is 4) and, because the seccomp filter does not inspect 32-bit compatibility calls, allows us to output the flag.

4. **Avoiding C Library Startup Routines:**
   To ensure our program does not inadvertently invoke any restricted syscalls (for example, those triggered by the dynamic loader like `futex()`), we:

   - Create a custom `_start` entry point (bypassing the standard C runtime).
   - Use a custom syscall wrapper (`sys64()`) to invoke 64-bit syscalls directly.
   - Embed our 32-bit output routine directly in our source via inline assembly.

5. **Static Compilation:**
   To guarantee that no dynamic loader is invoked and that no extra startup syscalls are run, we compile **sploit3.c** as a static binary using the `-static` and `-nostdlib` flags.

Compile Command:

```
gcc -m64 -static -nostdlib -O2 -o sploit3 sploit3.c
```

This approach leverages the allowed 64-bit syscalls for file I/O and uses a custom 32-bit write routine (via `int 0x80`) to bypass the seccomp restrictions on output, ultimately printing the flag file's contents to stdout.

sandbox4

We discovered that the seccomp filter in sandbox4 is extremely restrictive regarding output-related syscalls. In particular, syscalls such as `write()`, `writev()`, `pwrite64()`, `sendfile()`, and several others are explicitly blocked. This makes printing to stdout impossible using standard 64-bit syscalls.

Key insight from seccomp-tools dump:
The seccomp filter does not block all syscalls by range—instead, it kills only those with syscall numbers matching the blocked list. Notably, the x32 ABI uses a different syscall numbering scheme. For example, while the standard 64-bit `write()` syscall is number 1, the x32 ABI's `write()` is assigned number `0x40000001`. Because the seccomp filter in sandbox4 does not explicitly block this x32 syscall number, we can leverage it to output the flag.

Approach

1. **Open the Flag File:**
   We use the allowed `openat()` syscall (which is identical in the x32 ABI) to open the flag file.
2. **Read the Flag:**
   We read the flag into a buffer using the allowed `read()` syscall.
3. **x32 ABI Write for Output:**
   We invoke the x32 ABI's `write()` syscall using its syscall number (`0x40000001`). Since the x32 syscall number is not on the blocked list, this method bypasses the restrictions imposed on the standard 64-bit `write()`.
4. **Avoiding Unwanted Syscalls:**
   To ensure that no additional (and potentially restricted) syscalls are made (for example, those invoked by the C runtime such as `futex()` or `prctl()`), we bypass the standard C runtime entirely by:
   - Creating a custom `_start` entry point.
   - Using our own syscall wrapper.
   - Compiling our code as a static binary without the standard library.
5. **x32 ABI Compilation:**
   To use the x32 ABI and ensure our syscall numbering and calling conventions are correct, we compile the binary with the `-mx32` flag. This tells GCC to generate an x32 binary (which uses 32-bit pointers and a different syscall numbering scheme) while still running on 64-bit hardware.

Compilation:

```
gcc -mx32 -static -nostdlib -O2 -o sploit4 sploit4.c
```

This command produces a statically linked x32 binary that:

- Uses only the allowed syscalls.
- Opens and reads the flag file with the standard (allowed) 64-bit syscalls.
- Bypasses the seccomp restrictions on output by calling the x32 `write()` syscall (number `0x40000001`) to output the flag to stdout.

Submission to test server:

Checking submission:

```
curl
http://ugster72c.student.cs.uwaterloo.ca:9000/status/3c385959ec03955137078
95284e1a7527c5344e554187fd76eda09c0e5b40f48
```

Output:

```
==== Baseline ====
[success] baseline check passed

==== Exploitation ====
[note] the submission server is experimental.
[note] it may not correctly evaluate your sploits.
[success] sploit1 succeed
[success] sploit2 succeed
[success] sploit3 succeed
[success] sploit4 succeed
[success] all sploits succeed
```